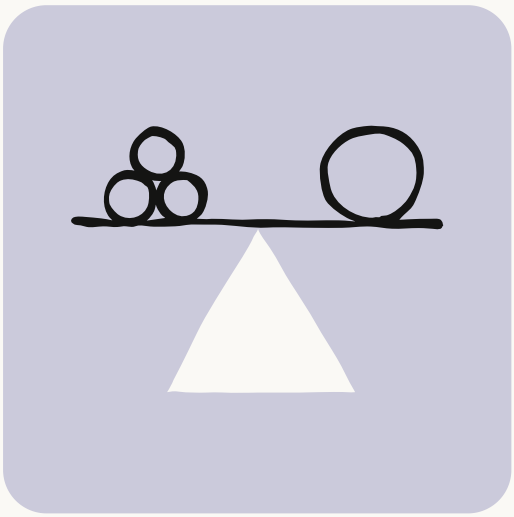


Building multi-agent systems: When and how to use them

While single-agent systems handle most enterprise workflows effectively, multi-agent architectures can unlock additional value for your organization. Learn when and how to use them.

Category
Agents

Product
Claude Platform
Claude Code

Date
January 23, 2026

Reading time
5 min

Share

A multi-agent system is an architecture where multiple LLM instances run with separate conversation contexts, coordinated through code. Multiple coordination patterns exist (agent swarms, capability-based systems, and message bus architectures), but this article focuses on the orchestrator-subagent pattern: a hierarchical model where a lead agent spawns and manages specialized subagents for specific subtasks. This pattern offers a straightforward coordination model and is a good starting point for teams new to multi-agent systems. We'll explore other patterns in detail in our next article.

Today, multi-agent systems are often applied in situations where a single agent would perform better, though this calculus continues to evolve as models improve. At Anthropic, we've seen teams invest months building elaborate multi-agent architectures only to discover that improved prompting on a single agent achieved equivalent results.

After building multi-agent systems and working with teams deploying them in production, we've identified three situations where multiple agents consistently outperform a single agent: when context pollution degrades performance, when tasks can run in parallel, and when specialization improves tool selection or task focus. Outside these situations, the coordination costs typically exceed the benefits.

In this article, we share how to recognize single-agent limits, identify the three scenarios where multi-agent systems excel, and avoid common implementation mistakes.

Multi-agent systems introduce overhead. Every additional agent represents another potential point of failure, another set of prompts to maintain, and another source of unexpected behavior.

We've observed teams build elaborate multi-agent systems with separate agents for planning, execution, review, and iteration, only to discover that they suffered from lost context at each handoff and spent more tokens coordinating than executing. In our testing, multi-agent implementations typically use 3-10x more tokens than single-agent approaches for equivalent tasks. This overhead stems from duplicating context across agents, coordination messages between agents, and summarizing results for handoffs.

A decision framework for multi-agent systems

Multi-agent architectures provide value when they address specific constraints that a single agent cannot overcome. This means multi-agent architectures should be reserved for cases where they provide clear benefits that justify the additional cost.

The patterns below represent cases where we consistently observe positive returns on this investment.

Context protection

Large language models have finite context windows, and response quality can degrade as context grows. When an agent's context accumulates information from one subtask that is irrelevant to subsequent subtasks, context pollution occurs. Subagents provide

history while diagnosing technical issues. If every order lookup adds thousands of tokens to the context, the agent's ability to reason about the technical problem degrades.

The single-agent approach:

```
# Single agent accumulates everything in context
conversation_history = [
    {"role": "user", "content": "My order #12345
isn't working"},
    {"role": "assistant", "content": "Let me check
your order..."},
    # Tool result adds 2000+ tokens of order history
    {"role": "user", "content": "... (order details, past
purchases, shipping info) ..."},
    {"role": "assistant", "content": "Now let me diagnose
the technical issue..."},
    # Context is now polluted with order details the agent
doesn't need
]
```

The agent must reason about the technical issue while maintaining 2000+ tokens of irrelevant order history in context, diluting attention and reducing response quality.

The multi-agent approach:

```
class OrderLookupAgent:
    def lookup_order(self, order_id: str) -> dict:
        # Separate agent with its own context
        messages = [
            {"role": "user", "content": f"Get essential
details for order {order_id}"}
        ]
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            messages=messages,
            tools=[get_order_details_tool]
        )
        # Returns only essential information
        return extract_summary(response)

class SupportAgent:
    def handle_issue(self, user_message: str):
        if needs_order_info(user_message):
            order_id = extract_order_id(user_message)
            # Get only what's needed, not full history
            order_summary =
OrderLookupAgent().lookup_order(order_id)
            # Inject compact summary, not full context
            context = f"Order {order_id}:
{order_summary['status']}, purchased
{order_summary['date']}"

        # Main agent context stays clean
        messages = [
            {"role": "user", "content": f"{context}\n\nUser
issue: {user_message}"}
        ]
```

```
        )
        return response
```

The order lookup agent processes the full order history and extracts a summary. The main agent receives only the 50-100 tokens it actually needs, keeping context focused.

Context isolation is most effective when subtasks generate high context volume (more than 1000 tokens) but most of that information is irrelevant to the main task, when the subtask is well-defined with clear criteria for what information to extract, and for lookup or retrieval operations that require filtering before use.

Parallelization

Running multiple agents in parallel allows you to explore a larger search space than a single agent can cover. This pattern has proven particularly valuable for search and research tasks.

Our Research feature uses this approach. A lead agent analyzes a query and spawns multiple subagents to investigate different facets in parallel. Each subagent searches independently, then returns distilled findings. Multi-agent search has shown substantial accuracy improvements over single-agent approaches by allowing exploration across larger information spaces.

The core implementation decomposes a question into independent facets, runs subagents concurrently, then synthesizes the results.

```
import asyncio
```

```
async def research_topic(query: str) -> dict:
    # Lead agent breaks query into research facets
    facets = await lead_agent.decompose_query(query)

    # Spawn subagents to research each facet in parallel
    tasks = [
        research_subagent(facet)
        for facet in facets
    ]
    results = await asyncio.gather(*tasks)

    # Lead agent synthesizes findings
    return await lead_agent.synthesize(results)

async def research_subagent(facet: str) -> dict:
    """Each subagent has its own context window"""
    messages = [
        {"role": "user", "content": f"Research: {facet}"}
    ]
    response = await client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=4096,
        messages=messages,
        tools=[web_search, read_document]
    )
    return extract_findings(response)
```

This improved coverage comes at a cost. Multi-agent systems typically consume 3 to 10 times more tokens than single-agent approaches for equivalent tasks. This happens because each agent needs its own context, agents must exchange messages to coordinate, and results must be summarized when passed between

The primary benefit of parallelization is thoroughness, not speed. When you need to search across a large information space or investigate many angles of a complex question, parallel agents can cover more ground than a single agent working within its context limits. The tradeoff is higher token usage and often longer total execution time in exchange for more comprehensive results.

Specialization

Different tasks sometimes benefit from different tool sets, system prompts, or domains of expertise. Rather than providing a single agent with access to dozens of tools, specialized agents with focused toolsets matched to their responsibilities can improve reliability.

Tool set specialization

When an agent has access to too many tools, performance suffers. Three signals indicate tool specialization would help:

1. **Quantity.** An agent with too many tools (often 20+) struggles to select the appropriate one.
2. **Domain confusion.** When tools span multiple unrelated domains (database operations, API calls, file system operations), the agent confuses which domain applies to a given task.
3. **Degraded performance.** Adding new tools degrades performance on existing tasks, suggesting the agent has reached its capacity for tool management.

System prompt specialization

following; a brainstorming agent needs creative flexibility. When a single agent must switch between conflicting behavioral modes, separating into specialized agents with tailored system prompts produces more consistent results.

Domain expertise specialization

Some tasks benefit from deep domain context that would overwhelm a generalist agent. A legal analysis agent might need extensive context about case law and regulatory frameworks. A medical research agent might need specialized knowledge about clinical trial methodology. Rather than loading all domain context into a single agent, specialized agents can carry focused expertise relevant to their specific responsibilities.

Example: Multi-platform integration. Consider an integration system where agents need to work across CRM, marketing automation, and messaging platforms. Each platform has 10-15 relevant API endpoints. A single agent with 40+ tools often struggles to select correctly, confusing similar operations across platforms. Splitting into specialized agents with focused toolsets and tailored prompts resolves selection errors.

```
from anthropic import Anthropic

client = Anthropic()

# Specialized agents with focused toolsets and
tailored prompts
class CRMAgent:
```

```
    opportunities, and account records. Always verify
record ownership
    before updates and maintain data integrity across
related records."""
    tools = [
        crm_get_contacts,
        crm_create_opportunity,
        # 8-10 CRM-specific tools
    ]

class MarketingAgent:
    """Handles marketing automation operations"""
    system_prompt = """You are a marketing automation
specialist. You
    manage campaigns, lead scoring, and email
sequences. Prioritize
    data hygiene and respect contact preferences."""
    tools = [
        marketing_get_campaigns,
        marketing_create_lead,
        # 8-10 marketing-specific tools
    ]

class OrchestratorAgent:
    """Routes requests to specialized agents"""
    def execute(self, user_request: str):
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            system="""You coordinate platform integrations.
Route requests to the appropriate specialist:
- CRM: Contact records, opportunities, accounts,
sales pipeline
```

```
                { role : "user", content : user_request}
            ],
            tools=[delegate_to_crm, delegate_to_marketing,
delegate_to_messaging]
        )
        return response
```

This pattern mirrors effective professional collaboration, where specialists with tools matched to their roles collaborate more effectively than generalists attempting to maintain expertise across all domains. However, specialization introduces routing complexity. The orchestrator must correctly classify requests and delegate to the right agent, and misrouting leads to poor results. Maintaining multiple specialized agents also increases prompt maintenance overhead. Specialization works best when domains are clearly separable and routing decisions are unambiguous.

Outgrowing single-agent architectures

Beyond the general framework, certain concrete signals suggest that single-agent patterns have been outgrown:

Approaching context limits. If an agent routinely uses large amounts of context and performance is degrading, context pressure may be the bottleneck. Note that recent advances in context management (such as compaction) are reducing this limitation, allowing single agents to maintain effective memory across much longer horizons.

Managing many tools. When an agent has 15-20+ tools, the model spends significant context and attention understanding its options.

Parallelizable subtasks. When tasks naturally decompose into independent pieces (research across multiple sources, tests for multiple components), parallel subagents can provide substantial speedups.

These thresholds will shift as models improve. Current limits represent practical guidelines, not fundamental constraints.

Context-centric decomposition

When adopting a multi-agent architecture, the most important design decision is how to divide work between agents. We've observed that teams frequently make this choice incorrectly, leading to coordination overhead that negates the benefits of multi-agent design.

The key insight is to adopt a **context-centric view** rather than a problem-centric view when decomposing work.

Problem-centric decomposition (often counterproductive). Dividing by type of work (one agent writes features, another writes tests, a third reviews code) creates constant coordination overhead. Each handoff loses context. The test-writing agent lacks knowledge of why certain implementation decisions were made and the code reviewer lacks the context of exploration and iteration.

Context-centric decomposition (usually effective). Dividing by context boundaries means an agent handling a feature should also handle its tests, because it already possesses the necessary context. Work should only be split when context can be truly isolated.

by software development role (planner, implementer, tester, reviewer), the subagents spent more tokens on coordination than on actual work.

Effective decomposition boundaries include:

- **Independent research paths.** Investigating "market trends in Asia" versus "market trends in Europe" can proceed in parallel with no shared context.
- **Separate components with clean interfaces.** With a well-defined API contract, frontend and backend work can proceed in parallel.
- **Blackbox verification.** A verifier that only needs to run tests and report results does not require implementation context.

Problematic decomposition boundaries include:

- **Sequential phases of the same work.** Planning, implementation, and testing of the same feature share too much context.
- **Tightly coupled components.** Components requiring constant back-and-forth belong in the same agent.
- **Work requiring shared state.** Agents that would need to frequently synchronize understanding should remain together.

The verification subagent pattern

One multi-agent pattern that consistently works well across domains is the **verification subagent**. This is a dedicated agent whose sole responsibility is testing or validating the main agent's work.

verification requires specialized tools, or when you want to enforce explicit verification checkpoints in your workflow.

Verification subagents succeed because they sidestep the telephone game problem. Verification requires minimal context transfer by nature, so a verifier can blackbox-test a system without needing the full history of how it was built.

Implementation

The main agent completes a unit of work. Before proceeding, it spawns a verification subagent with the artifact to verify, clear success criteria, and tools to perform verification.

The verifier does not need to understand why the artifact was built as it was. It only needs to determine whether the artifact meets the specified criteria.

```
from anthropic import Anthropic

client = Anthropic()

class CodingAgent:
    def implement_feature(self, requirements: str) -> dict:
        """Main agent implements the feature"""
        messages = [
            {"role": "user", "content":
f"Implement: {requirements}"}
        ]
        response = client.messages.create(
            model="claude-sonnet-4-5",
```

```
        return {
            "code": response.content,
            "files_changed": extract_files(response)
        }

class VerificationAgent:
    def verify_implementation(self, requirements: str,
files_changed: list) -> dict:
        """Separate agent verifies the work"""
        messages = [
            {"role": "user", "content": f"""
Requirements: {requirements}
Files changed: {files_changed}

Run the test suite and verify:
1. All existing tests pass
2. New functionality works as specified
3. No obvious errors or security issues

You MUST run the complete test suite before marking
as passed.
Do not mark as passing after only running a few tests.
Run: pytest --verbose
Only mark as PASSED if ALL tests pass with no failures.
"""}
        ]
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=4096,
            messages=messages,
            tools=[run_tests, execute_code, read_file]
        )
        return {
```

```
def implement_with_verification(requirements: str,
max_attempts: int = 3):
    for attempt in range(max_attempts):
        result =
CodingAgent().implement_feature(requirements)
        verification =
VerificationAgent().verify_implementation(
            requirements,
            result['files_changed']
        )

        if verification['passed']:
            return result

        requirements += f"\n\nPrevious attempt failed:
{verification['issues']}"

    raise Exception(f"Failed verification after
{max_attempts} attempts")
```

Applications

Verification subagents are effective for:

- **Quality assurance.** Running test suites, linting code, validating outputs against schemas.
- **Compliance checking.** Verifying documents meet policy requirements, checking outputs against rules.
- **Output validation.** Confirming generated content meets specifications before delivery.

The early victory problem

The most significant failure mode for verification subagents is marking outputs as passing without thorough testing. The verifier runs one or two tests, observes them pass, and declares success.

Mitigation strategies include:

- **Concrete criteria.** Specify "Run the full test suite and report all failures" rather than "make sure it works."
- **Comprehensive checks.** Require the verifier to test multiple scenarios and edge cases.
- **Negative tests.** Direct the verifier to attempt inputs that should fail and confirm they do.
- **Explicit instructions.** The instruction "You MUST run the complete test suite before marking as passed" is essential. Without explicit requirements for comprehensive validation, verification agents take shortcuts.

Moving forward

Multi-agent systems are powerful, but not universally appropriate. Before adding the complexity of multiple coordinated agents, confirm that:

1. **Genuine constraints exist** that multi-agent solves, such as context limits, parallelization opportunities, or need for specialization.
2. **Decomposition follows context, not problem type.** Group work by what context it requires, not by what kind of work it is.

complexity only when evidence supports it.

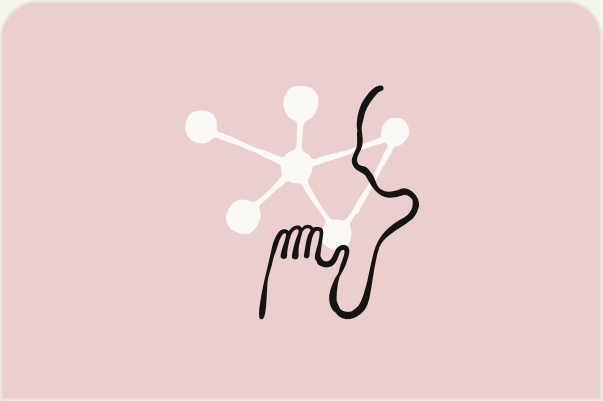
This is the first in a series of posts on multi-agent systems. For more on single-agent patterns, see Building effective agents. For context management strategies, see Effective context engineering for AI agents. For a deep dive into how we built our multi-agent research system, see How we built our multi-agent research system.

Acknowledgements

Written by Cara Phillips, with contributions from Paul Chen, Andy Schumeister, Brad Abrams, and Theo Chu.

Related posts

Explore more product news and best practices for teams building with Claude.



Apr 10, 2026

Multi-agent coordination patterns: Five approaches and when to use them

Agents

Apr 10, 2026

Preparing your security program for AI-accelerated offense

Agents

Apr 2, 2026

Harnessing Claude's intelligence

Agents

Mar 5, 2026

Common workflow patterns for AI agents—and when to use them

Agents

Transform how your organization operates with Claude

See pricing

Contact sales

Product updates, how-tos, community spotlights, and more. Delivered monthly to your inbox.

Enter your email

Claude

How can I help you today?

Write

Learn

Code